Pointer-First Agent Constitution: Architectural Audit and Refactoring Strategy for Stela Task Governance

1. Executive Summary: The Transition to Deterministic Governance

The evolution of repository-resident Artificial Intelligence (AI) has precipitated a fundamental crisis in software governance: the irreconcilable tension between the probabilistic fluidity of generative logic and the rigid, binary requirements of structural integrity. In the Stela ecosystem, this tension is not merely a theoretical concern but an operational bottleneck that threatens the viability of the platform. The current architecture, while structurally sound in its intent, suffers from a critical implementation flaw: reliance on narrative-driven instructions within the TASK.md surface.[1] This "Instruction Layer Bloat" forces agents to interpret governance as "Soft Law"—guidelines to be followed if context permits—rather than "Hard Physics"—immutable constraints enforced by the environment.

This report delivers an exhaustive forensic audit of the Stela repository, analyzing the specific friction points observed in recent agent "thinking logs" [1] and the structural anomalies present in the repository dump.[1] Our analysis confirms that the system is currently "Un-serviceable" due to three primary factors:

1. **Ghost Canon:** The persistence of outdated references (e.g., TRUTH.md) in automated workflows that conflict with the active constitution (PoT.md), creating a "Split-Brain" scenario for agents.[1]
2. **Context Hazards:** The inclusion of procedural logic within task definitions leads to "Token Saturation," degrading the agent's Signal-to-Noise Ratio (SNR) and causing critical instructions—such as the Mandatory Closing Block—to be ignored.[1]
3. **Cognitive Friction:** The lack of deterministic state pointers forces agents into "Hallucination Loops," specifically regarding branch creation and naming conventions, as evidenced by the hesitation in Gemini and ChatGPT logs.[1]

To resolve these systemic failures, this report proposes the immediate adoption of the **Pointer-First Agent Constitution**. This architectural paradigm shifts the locus of control from the prompt's context window to the repository's ops/ binary layer. By stripping TASK.md of redundant instructions and replacing them with executable pointers (e.g., replacing "Check for contractions" with tools/lint/style.sh), we transform the repository into a deterministic state machine. This report outlines the theoretical basis, forensic evidence, and concrete specifications for this transformation, ensuring that Stela achieves the high-entropy resilience

required for autonomous software development.

---

2. Theoretical Framework: The Physics of Pointer-First Architecture

To fully grasp the necessity of the Pointer-First architecture, one must first dissect the "Crisis of Context" inherent in Large Language Model (LLM) operations within code repositories. The legacy model of "Prompt Engineering"—stuffing context windows with elaborate behavioral instructions—is fundamentally flawed when applied to rigorous software engineering governance.

2.1 The Mechanics of Context Drift and Signal Degradation

"Drift" in the Stela environment is defined as the gradual divergence between the system's intended state, as defined by its governance documentation, and its actual operational state.[1] In legacy architectures, agents are instantiated with "thick" context windows loaded with paragraphs of behavioral instructions, coding guidelines, and project history. While intuitive, this approach suffers from a critical flaw identified in recent research on context engineering: as the context window expands with redundant text, the "Signal-to-Noise" (SNR) ratio diminishes.

In the specific case of Stela, this manifests as "Instruction Layer Bloat" within TASK.md. When an agent is presented with a 2,000-token prompt containing conflicting or repetitive instructions (e.g., "Read PoT.md" followed by a summary of PoT.md), the attention mechanism of the Transformer model may dilute the weight of the immutable "Hard Constraints".[1] This leads to "Ghost Canon"—a phenomenon where the agent follows the *flavor* of the instruction (e.g., "write clean code") rather than the *law* of the repository (e.g., "adhere to ops/lib/project/SCAFFOLD.md").

The Pointer-First Constitution addresses this by effectively eliminating the "instruction" layer from the agent's context. Instead of telling an agent *how* to behave (which is prone to interpretation), the architecture provides the agent with a pointer to an executable tool that *enforces* the behavior. This shifts the governance model from "requesting compliance" (probabilistic) to "preventing violation" (deterministic). By reducing the prompt to a sparse graph of pointers, we maximize the SNR, ensuring that the agent's cognitive resources are dedicated solely to payload execution rather than interpreting ambiguous governance text.

2.2 The Filing Doctrine as a Semantic Map

A critical component of the Pointer-First system is the "Filing Doctrine," which maps the repository tree to specific cognitive functions. This is not merely file organization; it is a "Semantic Map" that allows the agent to navigate the system without hallucinating structure.[1]

The doctrine establishes a rigid separation of concerns that must be enforced by the binary layer.

Directory	Semantic Definition	Cognitive Function	Operational Reality
ops/	**Run**	**Execution:** The "hands" of the system. Contains executable logic, binaries, and automation.	Agents execute tools here; they do not deliberate or edit unless explicitly tasked with platform engineering.
docs/	**Explain**	**Memory:** The "cortex" of the system. Contains manuals, rationale, and governance.	This is the source of truth (SSOT). Agents read here to understand *why*, but docs/ files effectively "do" nothing.
projects/	**Work**	**Output:** The "payload" where value is generated.	This is the only mutable surface for "Contractor" agents under normal DPs. It must be strictly isolated from platform logic.
storage/	**Trash**	**Ephemeral:** The "scratchpad." Explicitly excluded from canon.	Stores artifacts like handoff/ and dumps/. These prove work was done but do not define the rules of the work.

Legacy tasks that embed execution logic (which belongs in ops/) or lengthy explanations (which belong in docs/) into the TASK.md file violate this doctrine.[1] For example, if a task instruction says "Run git status and check for changes," it is embedding ops logic into a docs

surface. A Pointer-First TASK.md respects this separation by containing only the coordination logic necessary to link a specific objective to the correct ops/ tools and docs/ references.

2.3 The Axiom of Context Hygiene and JIT Loading

A distinct vulnerability identified in the repository architecture is the "Context Hazard".[1] The Policy of Truth (PoT.md) explicitly defines a Context Hazard as "any inclusion of docs/library/agents, docs/library/tasks, or docs/library/skills in the global context manifest is a failure".[1]

The global context (ops/lib/manifests/CONTEXT.md) is loaded at the start of every session via ops/bin/open. If the library of all possible tasks and skills were included here, two specific failures would occur:

1. **Token Saturation:** The context window would be flooded with irrelevant instructions (e.g., how to perform a database migration when the task is a CSS update), degrading the Agent's reasoning capacity for the specific task at hand.
2. **Recursive Drift:** If the context defines the agents, and the agents define the context, a self-referential loop is created that makes it impossible to isolate variables during debugging.[1]

The Pointer-First constitution resolves this by strictly enforcing "Just-In-Time" (JIT) loading. Agents, tasks, and skills are never loaded globally. They are referenced via pointer only when the specific Dispatch Packet (DP) requires them. This implies that TASK.md must be rigorously stripped of inline text; it must remain lightweight enough to be injected into the context on demand without displacing the actual project code.

---

3. Forensic Audit of the Current Stela Repository

A deep forensic audit of the provided repository dump (dump-full-main-eccc11128.txt) reveals significant discrepancies between the "Pointer-First" ideal and the current operational reality. These discrepancies serve as the primary friction points causing model hesitation and drift.

3.1 The "Ghost Canon" of TRUTH.md

The most glaring violation of "Referential Integrity" is the persistence of TRUTH.md in the automation layer despite PoT.md being declared the "single constitutional document".[1]

In the file .github/workflows/sop_policing.yml, the automation logic for enforcing "Canon" changes includes a check for a file named TRUTH.md:

JavaScript

```
// "Canon" = anything that changes project meaning...
const canonChanged = names.some(n =>
  n === "TRUTH.md" ||
  n === "TASK.md" ||
  n.startsWith("docs/")...
);
```

However, the **Policy of Truth (PoT.md)**, which serves as the "single constitutional document for Stela governance," lists the canonical constitution surface as PoT.md, not TRUTH.md. Furthermore, TRUTH.md does not appear in the repository's ``.[1] This indicates that TRUTH.md is an outdated rule or a "ghost" reference to a file that has been renamed to PoT.md.

This "Split-Brain" scenario forces the *Integrator* (Lead AI) or the automated gates to enforce a law that no longer exists. This is the definition of "Ghost Canon"—outdated rules lingering in the automation layer that compete with active governance. When an agent reads PoT.md, it sees one set of laws. When it encounters the workflow output or instructions referencing TRUTH.md, it sees another. This violation of the "Absolute Literalism" standard causes hesitation and drift.[1]

3.2 Instruction Layer Bloat in TASK.md

The current TASK.md file is heavily laden with redundant narrative instructions that violate the Pointer-First elimination rule. This redundancy is not a safety feature; it is a vector for entropy.

- **Redundant Context Loading:** Section 1 of TASK.md lists files that "Must read (always)" including PoT.md and SoP.md. It then requires the worker to echo "Loaded: PoT, SoP...". Section 3.2 repeats this exact requirement: "Worker must confirm loaded before acting.".[1]
    - *Forensic Impact:* This is "Instruction Layer Bloat." The requirement to load context should be enforced by the ops/bin/open binary, which generates the prompt. If the prompt is generated correctly, the context is loaded. Demanding a verbal echo is a weak, probabilistic check compared to a deterministic manifest verification. It consumes tokens and attention without adding verification value.
- **Jurisdictional Redundancy:** Section 3.3 repeats the "Worker Git Authority" rules (no branch creation, no pushes). These rules are already constitutional law in PoT.md Section 4.1. Repeating them in every task file invites "Drift" if PoT.md is updated but TASK.md is not. A Pointer-First task should simply state: "Jurisdiction: See PoT.md Section 4.1."
- **Procedural Bloat:** Section 3.4.5 lists a multi-step workflow for "Skill capture," including specific grep commands. This logic belongs in ops/lib/scripts/skill.sh or docs/library/SKILLS.md. Hardcoding it into TASK.md ensures that if the skill harvesting process changes, every active task becomes obsolete.[1]

3.3 The Discrepancy in Read-in Order

The .github/copilot-instructions.md file defines a specific **Read-in Order** for Guest Contractors that further complicates the context landscape:

1. PoT.md
2. SoP.md
3. TASK.md
4. docs/INDEX.md (Navigation)

This conflicts with **PoT.md Section 1.3 (Canon Surfaces)**, which explicitly defines the canonical wayfinding document as docs/MAP.md. While docs/INDEX.md exists, PoT.md identifies docs/MAP.md as the official surface for continuity. Identifying INDEX.md as a "Source of Truth" in the Copilot instructions creates a secondary "Ghost Canon" hazard where rules might be inferred from an unauthorized navigation file.[1]

3.4 The docs/ vs. ops/ Boundary Violation

The SAMPLE.pdf analysis highlights that docs/ is for explanation and ops/ is for execution. However, the audit reveals potential bleed-over. The existence of docs/ops/specs/binaries/ suggests a clean separation, but the presence of complex logic descriptions inside TASK.md (which functions as a doc) rather than ops/ scripts creates a "Parallel Jurisdiction."

For instance, the "Freshness Gate" instructions in TASK.md require the agent to manually run git rev-parse. This is a *manual* execution of what should be an automated check within ops/bin/open or tools/verify.sh. By asking the agent to manually verify the git state, we introduce the possibility of human (or AI) error. A Pointer-First system would simply require the agent to run tools/verify.sh --freshness, which would return a binary pass/fail based on the OPEN artifact.[1]

---

4. Cognitive Friction Analysis: Why Agents Hesitate

By analyzing the "Thinking Logs" of Gemini and ChatGPT [1], we can identify the specific cognitive friction points caused by the current architectural flaws. These logs serve as a psychological autopsy of the agents' decision-making processes, revealing why they stop, hesitate, or fail to complete tasks.

4.1 The Branch Creation Paradox

Both models displayed significant hesitation regarding the "Work Branch" name, creating a critical bottleneck at the start of the task.

- **The Conflict:** The TASK.md template requires a "Required Work Branch" field. The dump/OPEN file provided the "Base HEAD" (eccc11128) and "Base Branch" (main), but did

not specify a pre-existing work branch.[1]

- **The Operator Authority:** The "Staffing Protocol" in PoT.md explicitly states that the Operator (Human) "performs all commits, pushes, and merges" and implicitly owns branch creation. The models correctly identified that they (Contractors) are forbidden from creating branches.
- **The Logic Loop:** The models were trapped. They could not fill the "Required Work Branch" field because it didn't exist in the artifacts, but they couldn't create the branch because they lacked authority. ChatGPT considered proposing a name (work/agent-constitution-hardening-0039) but hesitated, fearing a "Not provided" placeholder would fail the dp lint tool.[1]
- **Root Cause:** The TASK.md asks the agent to *declare* state (the branch name) rather than *observe* state. In a Pointer-First system, the agent should never be asked to generate a branch name; it should only be asked to confirm that ops/bin/open has already generated one.

4.2 The "Mandatory Closing Block" Blind Spot

The prompts indicate that models frequently miss the "Mandatory Closing Block" [1], or fail to include it entirely.

- **The Cause:** The thinking logs show the models focusing heavily on the "Execution Plan" (Section 3.4) and the "Freshness Gate" (Section 3.1). The "Closing Block" (Section 4) is located at the very end of a long, text-heavy file (TASK.md).
- **Token Recency Bias:** In long context windows saturated with redundant instructions ("Instruction Layer Bloat"), instructions at the very end are deprioritized. The agent exhausts its "planning tokens" on the complex execution steps and treats the closeout as a generic administrative footer rather than a strict content requirement.
- **The Miss:** Gemini's log shows it prioritizing "Stress Testing" and "Refactoring" implementation steps.[1] It treats the "Closeout" as a "To-Do" item rather than a "Must-Output" item.
- **Root Cause:** The Closing Block is defined as a set of narrative instructions ("Varied Wording Required") rather than a pointer to a template. If TASK.md simply said "Output ops/lib/templates/CLOSEOUT.md filled with session data," the agent would treat it as a form-filling exercise (deterministic) rather than a creative writing task (probabilistic).

4.3 Linter Fear and Placeholder Paralysis

ChatGPT's thinking process revealed a specific fear of the dp lint tool.[1] It recognized that putting "Not provided" in a required field would cause a linter failure. This anxiety caused it to halt and consider asking the user for input.

- **Insight:** This proves the effectiveness of "Preventative Governance." The mere existence of the linter (even if not run) shaped the agent's behavior. The agent simulated the linter's execution and modified its plan to avoid failure.

- **Implication:** We must double down on linter-based governance. If the linter is the only thing the agent fears, then all rules must be encoded in linters. The hesitation was correct; the system *should* stop if a branch is missing. The failure was in the *workflow* that didn't provide the branch, not the agent's hesitation.

---

5. Refactoring Strategy: The Pointer-First TASK.md

Based on the audit and cognitive analysis, we propose a radical refactoring of TASK.md. This new schema strips away narrative instruction and replaces it with pointer logic, creating a "sparse graph" that maximizes the Signal-to-Noise Ratio.

5.1 The New TASK.md Schema

The new TASK.md acts as a **Dashboard**, not a **Manual**. It does not teach; it tracks.

Section 1: The Freshness Pointer (Replacing "Required Context Load")

Instead of listing files to read, Section 1 now points to the OPEN artifact generated by ops/bin/open.

STELA TASK DASHBOARD (LIVING SURFACE)

Status: ACTIVE

Owner: Integrator

Last Updated: {{TIMESTAMP}}

1. Session State (The Anchor)

Pointer: storage/handoff/OPEN-*.txt (The generated session context)

- **Active Branch:** {{BRANCH_NAME}} (Must match OPEN artifact)
- **Base HEAD:** {{SHORT_HASH}} (Must match OPEN artifact)
- **Context Manifest:** ops/lib/manifests/CONTEXT.md (Checked by tools/lint/context.sh)

Rationale: This removes the redundancy of listing files. The OPEN artifact *is* the list. The agent is forced to look at the machine-generated state, eliminating the "Branch Creation Paradox" by grounding the agent in what *exists* rather than what is *requested*.

Section 2: The Logic Pointers (Replacing "Behavioral Standards")

Instead of repeating "No contractions" or "No direct pushes," we use a ## Pointers section that references the binary gates.

2. Logic Pointers (The Law)

Primary Constraint: PoT.md (Policy of Truth) wins in all conflicts.

2.1 Governance Pointers

- **Jurisdiction:** PoT.md Section 3.
- **Git Authority:** PoT.md Section 4.1 (Operator owns pushes).
- **Behavioral Standard:** PoT.md Section 4.2.

2.2 Execution Pointers (The Toolchain)

- **Linguistic Precision:** tools/lint/style.sh (Enforces no contractions).
- **Structure Verification:** tools/verify.sh (Enforces Filing Doctrine).
- **Context Hygiene:** tools/lint/context.sh (Enforces manifest compliance).
- **Truth Integrity:** tools/lint/truth.sh (Enforces canon spelling).

Rationale: This creates the "Pointer-First" entity. The agent knows *where* the law is (PoT.md) and *how* it is enforced (tools/lint/*.sh). It does not need to be told "Don't use contractions"; it knows style.sh will kill the process if it does.

Section 3: The Dispatch Packet (Refactored for Determinism)

The DP section is stripped of procedural text.

3. Current Dispatch Packet (DP)

DP-OPS-XXXX: {{TITLE}}

3.1 Scope Boundary

Objective: {{ONE_SENTENCE_GOAL}}

Allowlist (Hard Gate):

- tools/lint/style.sh
- tools/verify.sh
- (new) tools/test/agent.sh

Stop Condition: Any edit outside the Allowlist is a **SYSTEM FAILURE**. STOP immediately.

3.2 Execution Plan

1. **Context Load:** Verify OPEN artifact matches git status.

2. **Execution:** Apply changes to Allowlist files.
3. **Verification:** Run ops/bin/dump and tools/verify.sh.
4. **Receipt:** Generate storage/handoff/DP-XXXX-RESULTS.md.

3.3 Skill Integration

Harvest/Promote: Use ops/lib/scripts/skill.sh.

- **Constraint:** Do not manually edit docs/library/SKILLS.md. Use the script.

Rationale: The "Scope Boundary" is now a "Hard Gate" tied to the file system. The "Execution Plan" is simplified to high-level steps that point to scripts. The "Skill Integration" explicitly forbids manual editing, closing the "Ghost Canon" loop.

Section 4: The Mandatory Closing Block (Template-Based)

The closing block is formalized to ensure the "Varied Wording" requirement is met by design.

4. Closeout (Mandatory)

Requirement: Populate storage/handoff/DP-XXXX-RESULTS.md and append the following block.

Mandatory Closing Block

> **Constraint:** "Final Squash Stub" must differ significantly from "Primary Commit Header".

1. **Primary Commit Header:** (Plaintext)
2. **Pull Request Title:** (Plaintext)
3. **Pull Request Description:** (Markdown)
4. **Final Squash Stub:** (Plaintext - Varied)
5. **Extended Technical Manifest:** (Plaintext list of files)
6. **Review Conversation Starter:** (Markdown question for Operator)

Rationale: Explicitly stating the constraint ("Must differ significantly") right next to the field leverages the "Constraint Satisfaction" capabilities of the LLM. It transforms the requirement from a forgotten preamble instruction into an immediate, form-filling constraint.

---

6. Implementation Plan: The Migration to Pointer-First

To transition Stela to this new constitution, we must execute a multi-phase "Refactoring Operation" that cleanses the repository of "Ghost Canon" and "Instruction Bloat."

Phase 1: The Registry Cleanse (Day 1)

- **Objective:** Eliminate "Ghost Canon" (TRUTH.md references) and align PoT.md as the supreme authority.
- **Action 1:** Search and Replace TRUTH.md with PoT.md in .github/workflows/sop_policing.yml and .github/copilot-instructions.md.
- **Action 2:** Update .github/copilot-instructions.md to remove docs/INDEX.md from the "Read-in Order" and replace it with docs/MAP.md as per PoT.md Section 1.3.[1]
- **Action 3:** Verify that docs/GOVERNANCE.md contains no prescriptive rules, only pointers to PoT.md. If it contains rules, migrate them to PoT.md or delete them.

Phase 2: The TASK.md Refactor (Day 2)

- **Objective:** Replace the current TASK.md with the "Pointer-First" schema defined in Section 5.1.
- **Action 1:** Archive the current TASK.md to storage/archives/.
- **Action 2:** Commit the new TASK.md template.
- **Action 3:** Update ops/bin/open to populate the new Section 1: Session State automatically. This resolves the "Branch Creation Paradox" by having the machine (Operator) define the state before the Agent (Contractor) sees it.

Phase 3: Toolchain Alignment (Day 3)

- **Objective:** Ensure the binaries referenced in the new TASK.md actually exist and enforce the rules.
- **Action 1:** Update tools/lint/style.sh to strictly enforce "Contraction Prohibition" on markdown files (as requested in DP-SAMPLE).[1]
- **Action 2:** Update tools/verify.sh to fail if docs/ contains binaries or ops/ contains loose markdown, enforcing the "Filing Doctrine".[1]
- **Action 3:** Create tools/lint/dp.sh (if not robust) to validate that the new TASK.md format is preserved during DPs.

Phase 4: Certification (Day 4)

- **Objective:** Stress test the new system.
- **Action:** Run the "Drift Injection" test.[1] Manually introduce a TASK.md change that violates PoT.md (e.g., "Use contractions"). Run the new tools/lint/style.sh and verify it fails.
- **Action:** Run the "Branch Paradox" test. Attempt to start a session without an OPEN artifact. Verify the Agent halts immediately because it cannot fill "Section 1: Session State."

---

7. Deep Dive: Addressing the "Branch Name

Hesitation"

The cognitive analysis of ChatGPT and Gemini [1] revealed a critical flaw in the current workflow: the "Branch Name Hesitation." Agents stopped because they were asked to provide a "Required Work Branch" that didn't exist in the OPEN artifact.

The Pointer-First Constitution solves this by strictly enforcing the **Staffing Protocol** via file structure.

The Old Workflow (Flawed):

1. Operator runs open.
2. Agent reads TASK.md.
3. TASK.md asks Agent: "What is the Work Branch?"
4. Agent looks at OPEN artifact. It only sees main.
5. Agent thinks: "I need a work branch. I can't create one. I can't invent one. STOP."

The Pointer-First Workflow (Corrected):

1. Operator runs ops/bin/open --branch=work/feature-01.
2. ops/bin/open creates storage/handoff/OPEN...txt containing "Branch: work/feature-01".
3. Agent reads TASK.md which points to OPEN...txt.
4. Agent reads "Branch: work/feature-01" from the artifact.
5. Agent proceeds.

This effectively moves the "Decision" of branch naming from the probabilistic Agent layer to the deterministic Operator layer (the open binary arguments). The Agent no longer needs to "think" about the branch name; it only needs to "read" it. This is the essence of "Pointer-First": **Don't ask the Agent to decide; ask the Agent to read the decision.**

---

8. The Mandatory Closing Block: A Mechanism for Entropy Control

The "Mandatory Closing Block" is not just paperwork; it is an entropy control mechanism. LLMs have a tendency towards "Pattern Collapse," where they repeat the same phrases (e.g., "Updated files") for every commit.

The requirement for "Varied Wording" (specifically that the "Final Squash Stub" must differ from the "Primary Commit Header") forces the model to engage its reasoning center at the very end of the task. By forcing it to re-describe the work in a new way, we ensure:

1. **Self-Correction:** The model often catches errors when forcing itself to re-summarize.
2. **Auditability:** The "Extended Technical Manifest" forces a final listing of touched files,

which the Integrator can compare against the git diff --stat.

Recommendation: The TASK.md must explicitly link this block to S-LEARN-07 (Harvest and Promote Skills). The closing block is the trigger for skill harvesting. If the block is incomplete, no skill is harvested, and the "institutional knowledge" of the repository stagnates.

---

9. Conclusion

The transition to a **Pointer-First Agent Constitution** is not merely a syntactic refactoring of TASK.md; it is a fundamental shift in the ontology of the Stela repository. We are moving from a system of "Soft Law," where rules are embedded in persuasive text, to a system of "Hard Physics," where rules are enforced by binary gates.

The forensic audit confirmed that the current system is "Un-serviceable" due to "Ghost Canon" (TRUTH.md), "Instruction Layer Bloat," and "Context Hazards." The cognitive logs of the agents confirmed that these structural flaws directly cause hesitation and failure.

By implementing the **Standardized Task Schema** defined in Section 5.1 and enforcing the **Filing Doctrine** via the toolchain updates in Section 6, Stela can eliminate "Context Drift." The agents will no longer need to "understand" the governance; they will simply be unable to violate it. In this deterministic environment, the "Integrator" and "Contractor" can operate with the precision of a compiler, ensuring that the repository remains a source of truth rather than a collection of probabilities.

Appendix A: Verified Data Tables

Table 1: Verified File Paths (Pointer-First)

File	Role	Verification Tool
PoT.md	Constitution (SSOT)	tools/lint/truth.sh
SoP.md	History Ledger	ops/bin/prune
TASK.md	Active Dashboard	tools/lint/task.sh
ops/bin/open	Session Generator	(Self-verified)
tools/verify.sh	Structure Guard	(Self-verified)

Table 2: "Ghost Canon" Remediation List

Ghost Artifact	Location Found	Remediation
TRUTH.md	.github/workflows/sop_policing.yml	Replace with PoT.md
docs/INDEX.md (as SSOT)	.github/copilot-instructions.md	Replace with docs/MAP.md
"Read PoT" (Text)	TASK.md Section 1	Replace with OPEN artifact pointer
Manual Git Checks	TASK.md Section 3.1	Replace with tools/verify.sh

Table 3: Mandatory Closing Block Specification

Field	Requirement	Validation Logic
Primary Commit Header	Plaintext, < 50 chars	wc -c
Pull Request Title	Plaintext	grep vs Commit Header
Final Squash Stub	**Must differ from Commit Header**	diff string comparison
Technical Manifest	List of files	Match git diff --name-only

Works cited

1. ChatGPT 5.2 Extended Thinking About Writing DP-SAMPLE.md